Monte Carlo Simulation

Estimation of π and Euler’s number e
Experiment record · 20 October 2026

Purpose and setup

The experiments estimate two known constants using repeated random sampling. The reference values allow the error to be measured, while five runs at each sample size show how much the estimates vary.

Setting	Value
Sample sizes	10, 100, 1,000, 10,000
Runs per sample size	5 for each experiment
Total estimates	40
Seed	20261019
Engine	Go PCG (math/rand/v2)

Method for π

For each run, n independent points (X, Y) are sampled uniformly in the unit square. A point is counted inside the quarter circle if $X^2 + Y^2 \leq 1$. The estimate is 4 times the inside count divided by n. This follows page 62 of the lecture notes.

Method for e

A trial starts with sum zero and count zero. Independent uniform numbers are added until the sum is strictly greater than 1. The number of draws is recorded. Repeating this trial n times and averaging the counts gives one estimate of e. Each sample size has five complete runs. This follows page 64 of the notes.

Source of the results

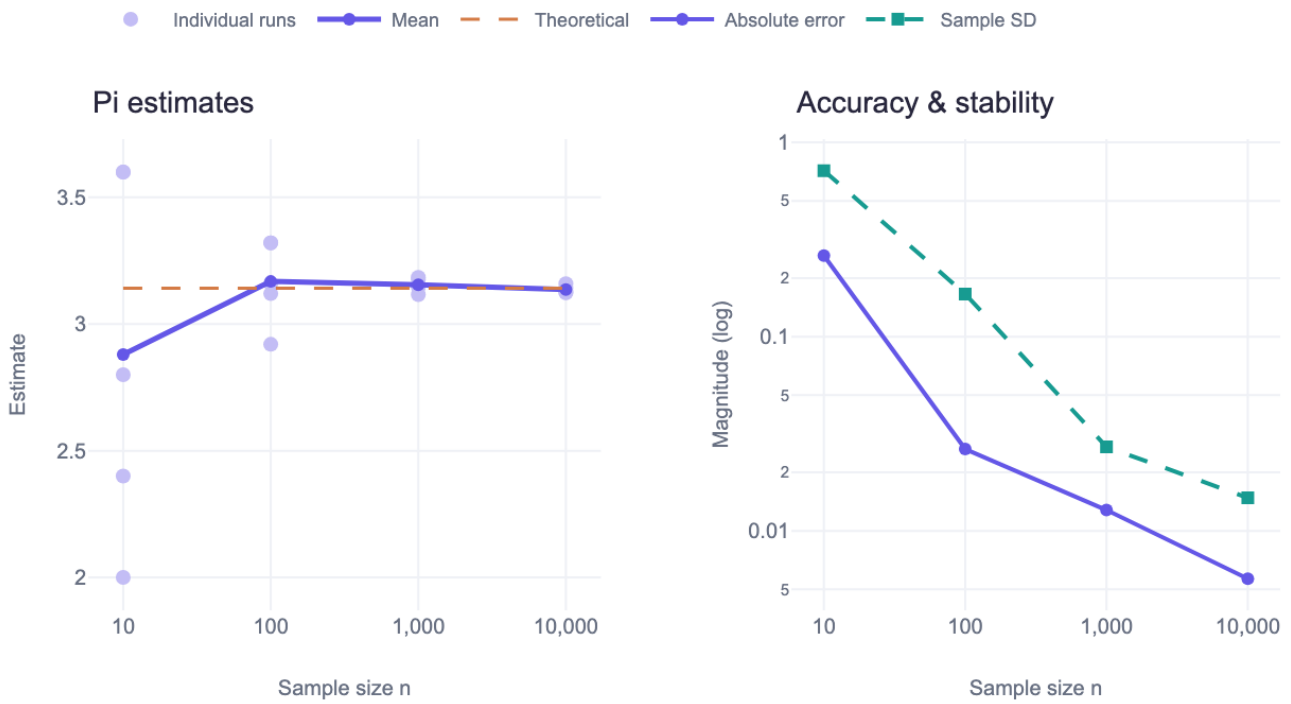
The application was started once with the settings above. Tables use its JSON export; the two figures were downloaded through the application’s PNG buttons. CSV, JSON, run log and original PNG files are included under results/2026-10-20. No results from another seed were mixed into this record.

This report and application were prepared with AI assistance. The supplied assignment prohibits AI use; this is not represented as independent student work. The PDF notes request Python, while the separately supplied instructions allow other languages. This implementation uses Go, with an optional Python launcher.

Pi experiment

n	Run 1	Run 2	Run 3	Run 4	Run 5	Mean	Reference
10	3.60000	2.80000	2.40000	3.60000	2.00000	2.88000	3.14159
100	3.32000	3.16000	2.92000	3.32000	3.12000	3.16800	3.14159
1000	3.18400	3.17600	3.14400	3.15200	3.11600	3.15440	3.14159
10000	3.14200	3.12320	3.12440	3.15880	3.13120	3.13592	3.14159

n	Mean	Absolute error	Sample SD
10	2.880000	0.261593	0.715542
100	3.168000	0.026407	0.165892
1000	3.154400	0.012807	0.027070
10000	3.135920	0.005673	0.014809



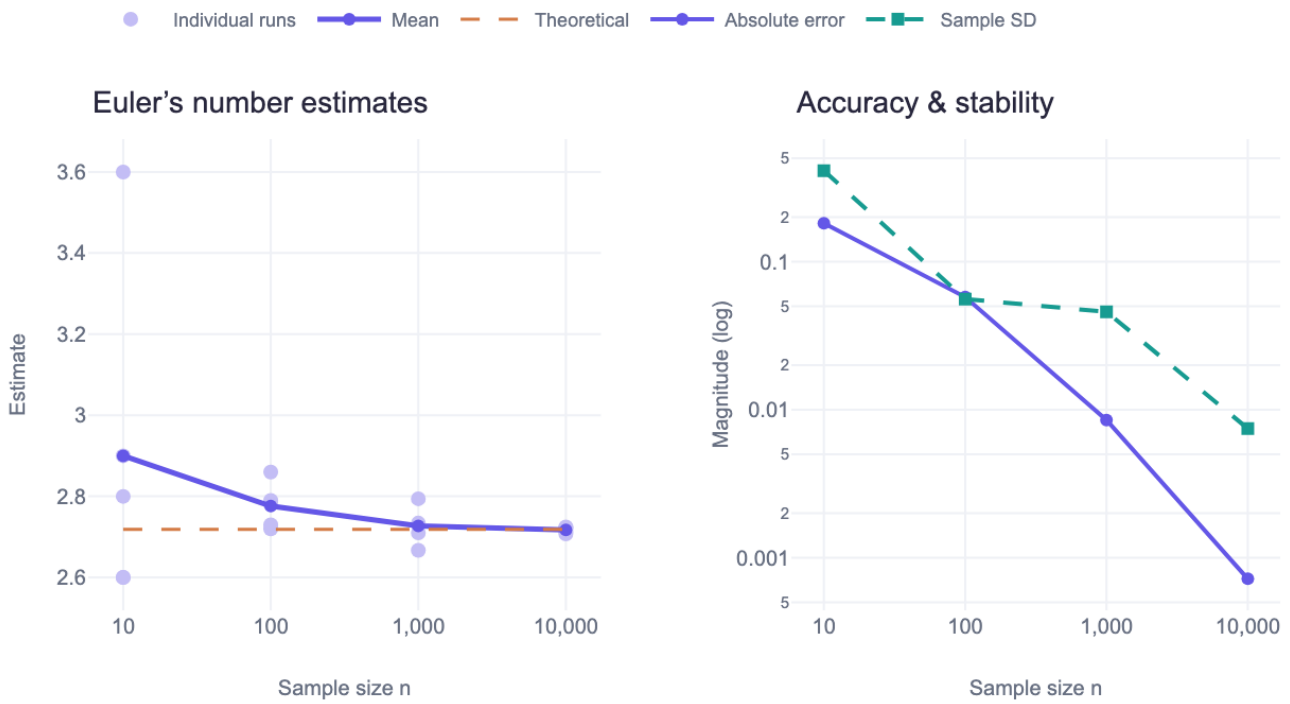
Application export. Left: all runs, mean and reference. Right: absolute error and sample SD. Sample size and right-hand magnitude axes are logarithmic.

The mean moved from 2.880000 at n=10 to 3.135920 at n=10,000. Absolute error decreased at every tested size in this run. The spread between runs also fell: sample SD changed from 0.715542 to 0.014809. The n=10 estimates ranged from 2.0 to 3.6, showing how strongly a small sample can vary.

Euler experiment

n	Run 1	Run 2	Run 3	Run 4	Run 5	Mean	Reference
10	2.80000	2.90000	3.60000	2.60000	2.60000	2.90000	2.71828
100	2.72000	2.79000	2.86000	2.78000	2.73000	2.77600	2.71828
1000	2.72900	2.79400	2.73400	2.66700	2.71000	2.72680	2.71828
10000	2.72180	2.72470	2.72180	2.71240	2.70710	2.71756	2.71828

n	Mean	Absolute error	Sample SD
10	2.900000	0.181718	0.412311
100	2.776000	0.057718	0.055946
1000	2.726800	0.008518	0.045910
10000	2.717560	0.000722	0.007466



Application export. Left: all runs, mean and reference. Right: absolute error and sample SD. Sample size and right-hand magnitude axes are logarithmic.

The mean moved from 2.900000 at n=10 to 2.717560 at n=10,000. Absolute error fell from 0.181718 to 0.000722. Sample SD fell from 0.412311 to 0.007466. The n=10 estimates ranged from 2.6 to 3.6; the larger sample produced a much tighter group.

Discussion

Which sample size had the greatest variation?

For both experiments, n=10 had the greatest sample SD: 0.715542 for π and 0.412311 for e. The smallest SD occurred at n=10,000. These statements describe this recorded run.

What changed as n increased?

Both absolute error and between-run SD decreased across the four tested sizes. This supports the expected improvement in accuracy and stability, but it does not guarantee the same monotonic pattern for another seed.

Why do runs with the same n differ?

Each run uses a different initialized pseudo-random stream. Different sampled points or uniform sequences give different estimates. Repeating the same complete settings reproduces the stored results.

How does the Law of Large Numbers apply?

For π, the average inside-circle indicator tends toward π/4; multiplying by four gives π. For e, the average count of draws needed to exceed 1 tends toward its expectation e. Larger within-run samples reduce typical sampling variation.

What would higher accuracy require?

Typical Monte Carlo standard error scales approximately as $1/\sqrt{n}$. Halving it generally requires about four times as many trials, so more computation is needed. This is a statement about typical error, not a guarantee for one estimate.

How the statistics were calculated

Mean is the average of the five estimates for one n. Absolute error is the distance from that mean to the full-precision reference constant. Sample SD uses the sum of squared deviations from the five-run mean divided by four, followed by the square root. SD is not a confidence interval. Tables show rounded numbers; calculations use the unrounded application data.

References

I-Stochastic Processes.pdf, Manisa Celal Bayar University, Industrial Engineering: p. 15 (Law of Large Numbers), p. 48 (simulation accuracy), pp. 61–66 (assignment methods, tables and discussion). Application source and dependency notices are included in the project package.

Yazılım ve uygulama bilgileri

Bu projede yazılım dili olarak Go (Golang) kullanılmıştır. Pi ve Euler sayısı için rastgele denemelerin yapılması, tekrarların yönetilmesi, ortalamaların ve hata değerlerinin hesaplanması Go tarafında gerçekleştirilir. Uygulama, bilgisayarda çalışan yerel bir web sunucusu üzerinden tarayıcıya açılır.

Projedeki odev.py dosyası yalnızca derlenmiş Go programını başlatır. Deneylerin hesabını Python yapmaz. Böylece aynı uygulama ister Python başlatıcısı üzerinden ister doğrudan çalıştırılabilir dosyadan açılabilir. Go tarafında üçüncü taraf bir hesaplama paketi kullanılmamıştır; işlemler standart kütüphane ile yürütülür.

Arayüz ve grafikler

Arayüz HTML, CSS ve JavaScript ile hazırlanmıştır. Sekmeler ve bazı hareketli bileşenlerde React, Base UI ve Framer Motion kullanılmış; UI TripleD bileşenleri uygulamaya uyarlanmıştır. Grafikler Plotly ile çizilir. Kullanıcı örneklem büyüklüğünü, tekrar sayısını ve seed değerini değiştirebilir; sonuçları tablo olarak inceleyebilir ve CSV, JSON veya grafik görseli olarak kaydedebilir.

Arayüz dosyaları Go programına gömüldüğü için hazır uygulama çalışırken internetten kütüphane indirilmez. Ön yüzü yeniden derlemek için esbuild kullanılır. Hazır paket macOS ARM64 içindir; başka işletim sistemlerinde kaynak koddan uygun bir çalıştırılabilir dosya oluşturulması gerekir.

Bu rapordaki veriler

Rapordaki tablolar, seed 20261019 ile yapılan tek uygulama çalışmasının JSON çıktısından hazırlanmıştır. Her iki deneyde n = 10, 100, 1000 ve 10000 değerleri için beşer tekrar vardır. Grafikler aynı çalışmadan, uygulamanın PNG indirme düğmeleriyle alınmıştır. Dosyalar GitHub deposundaki results/2026-10-20 klasöründe bulunur.

Proje bağlantısı

Proje sahibi: Doğukan Ercan · DengeX

https://github.com/dengexco/monte-carlo-studio

Kaynak kod, kullanım açıklamaları, kütüphane bilgileri ve deney çıktıları bu adreste yer alır. Hazır ZIP paketi ile raporun güncel sürümü aşağıdaki Releases bağlantısından indirilebilir.

GitHub üzerinden güncel paketi indir

Geliştirme notu: Proje ve belgeler AI yardımıyla hazırlanmıştır. Hesaplama dili Go’dur; Python yalnızca başlatıcıdır. Dersin programlama dili ve AI kullanım koşulları ayrıca geçerlidir.